

EyesOnGuj

A Unified, Interoperable CCTV Integration and Video Analytics Platform

High-Level Design Document

Technical Architecture, Data Model, Analytics Pipeline, Security and Scale Plan

Document version: 1.0

Date: September 2026

Repository: EyesOnGuj

Scope: Registry & GIS foundation, direct video ingestion and AI analytics, VMS federation middleware, and the statewide scale plan for CCTV integration across Gujarat's government departments.

This document describes the system as implemented in the accompanying codebase. Section 12 states plainly which capabilities are live today, which are partially built, and which are documented as near-term roadmap – consistent with how the rest of this project's engineering documentation already treats unfinished work.

Contents

1	Purpose and Scope	1
2	Architecture Overview	1
2.1	Why a hybrid model	1
2.2	System architecture	1
2.3	Deployment shell	2
3	Camera Onboarding and Integration Strategy	2
3.1	The adapter pattern	2
3.2	One camera table, three contributors	3
4	Registry and GIS Foundation	4
4.1	Data model	4
4.2	Coverage gap analysis	4
4.3	Role-based UI and audit trail	5
5	Unified Viewing and Direct Video Ingestion	5
5.1	Streaming architecture	5
5.2	Selective metadata, not bulk video	5
5.3	Dual ingestion: live and forensic	5
6	VMS Federation and Middleware	6
6.1	Event bus and correlation engine	6
6.2	No separate federation schema	6
7	AI Video Analytics Pipeline	6
7.1	Vehicle detection, tracking and ANPR	6
7.2	Biometric face-matching	7
7.3	Evaluation discipline	8
8	Watchlist Correlation and Real-Time Alerting	8
8.1	Data model	8
8.2	Severity and notification workflow	8
8.3	Why plate normalisation matters here specifically	9
9	Security, Privacy and Auditability	9
9.1	Authentication and session handling	9
9.2	Role-based access control	9
9.3	Login rate limiting	9
9.4	Audit trail	9
9.5	Sensitive media access	9
9.6	Transport security	10
9.7	What is documented here but not yet built	10
10	Deployment Architecture and Statewide Scale Plan	10
10.1	Current deployment	10
10.2	Path to statewide scale	11
10.3	Cost posture	11
10.4	Phased rollout	11
11	Assumptions and Information Required from Departments	12
12	Implementation Status	12

12.1 Testing approach	12
13 Appendix	12
13.1 Glossary	12
13.2 Data model overview	12

1 Purpose and Scope

This document is the technical architecture reference for EyesOnGuj, a platform for integrating heterogeneous, departmentally-owned CCTV infrastructure across Gujarat into one operational picture: a shared camera registry, unified live viewing, AI-driven vehicle and person analytics, and cross-VMS correlation against watchlist databases, without requiring any department to replace infrastructure it already runs and trusts.

It is written for anyone evaluating, extending, or operating the system: what exists, why it is built the way it is, where the architectural seams are, and what is intentionally deferred rather than silently missing. Section 12 is the single most load-bearing part of this document for that last point, and is worth reading before the rest.

The design responds to four structural facts about the problem: twenty-six departments run independent camera estates on different vendors, VMS platforms, storage backends and retention policies; camera sites are spread across the state, in some cases roughly a thousand kilometres apart; any useful analytics layer has to work uniformly across all of that heterogeneity; and the platform has to grow from a pilot of a few dozen cameras today toward a statewide estate on the order of eighty thousand, without a redesign along the way.

2 Architecture Overview

2.1 Why a hybrid model

EyesOnGuj does not commit to a single reference architecture. It combines three of them, because the three solve three different, genuinely distinct problems that a real statewide rollout will always have simultaneously:

- **Registry & GIS foundation.** Every camera, regardless of how it is eventually connected, needs one authoritative record: where it is, who owns it, what state it's in. This has to exist independently of any streaming or federation decision, and every other model builds on it.
- **Direct, middleware-free ingestion.** Where a camera or NVR is directly reachable over RTSP, the platform should connect straight to it. Adding a transcoding proxy or broker in front of a feed that doesn't need one is unnecessary latency, unnecessary bandwidth, and one more system that can fail.
- **Federation for vendor-locked and remote VMS estates.** A department running a closed VMS (a Milestone, HikCentral or Dahua-class platform) is not going to expose raw RTSP to an outside system, and shouldn't have to. That estate needs an adapter that speaks the VMS's own protocol, or the open ONVIF standard, or a generic REST API, and reports into the same correlation layer as everything else.

These aren't three competing proposals picked to hedge a scoring rubric; they are three answers to three different onboarding realities that all show up in the same statewide estate at once. A pilot deployment can lean almost entirely on direct RTSP because the demo cameras are reachable directly; a real statewide rollout will need the federation layer for the departments that don't expose raw streams. Building both from day one means the architecture doesn't need to change shape when that mix shifts.

2.2 System architecture

Figure 1 shows the system end to end: four classes of camera source, two onboarding paths into the platform, one shared application core, one shared data layer, and one set of operator-facing dashboards drawing on all of it.

The load-bearing decision in this diagram is that both ingestion paths feed the *same* application

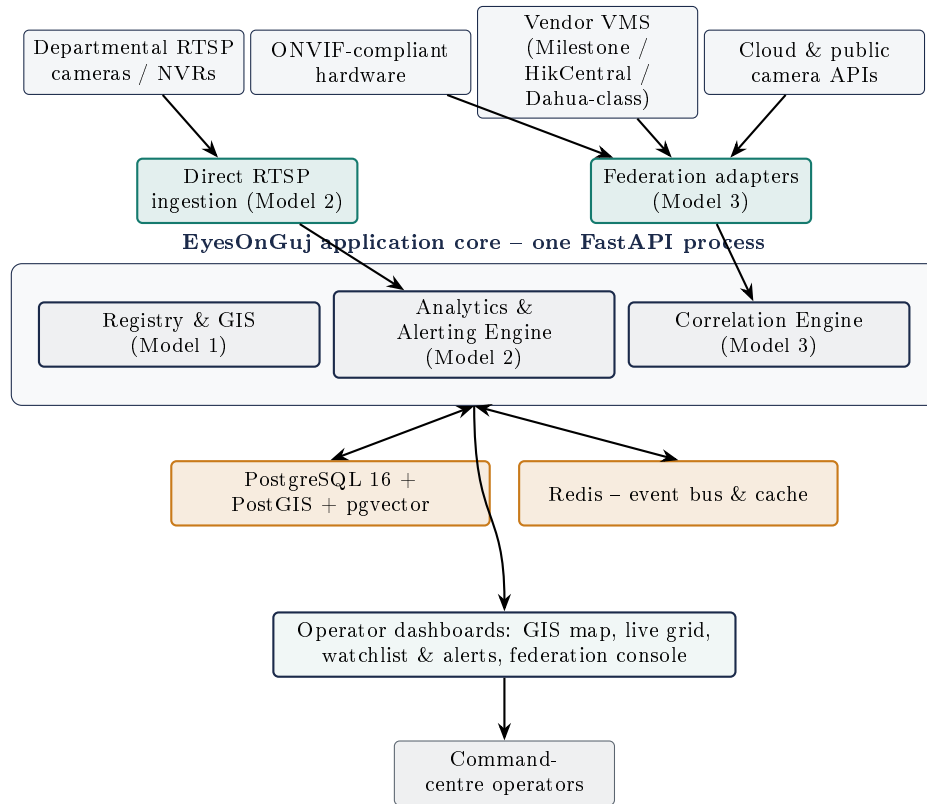


Figure 1: System architecture: sources, ingestion paths, application core, data layer, and operator dashboards.

core, the same PostgreSQL schema, and the same correlation logic. A vehicle detection from a directly-ingested RTSP feed and a detection event relayed through a federated VMS adapter land in the same `detections` table, get correlated against the same watchlist, and can be joined into the same cross-camera route. There is no "federated data" silo sitting apart from "direct" data; a camera is a camera and a sighting is a sighting, regardless of which door it came in through.

2.3 Deployment shell

All three models run inside one FastAPI application process rather than as three separate services. This is a deliberate simplification, not a shortcut: it means one login session, one role-based access control layer, one template engine, and one deployment unit, instead of three services that all have to independently agree on how authentication works. The trade-off, and it is a real one, is that scaling the application means scaling one process rather than scaling each concern independently; Section 10 addresses how that trade-off is managed as camera count grows.

3 Camera Onboarding and Integration Strategy

Heterogeneity is the first-order problem this platform exists to solve, and it is treated as a routing decision rather than a special case handled ad hoc per vendor. Table 1 summarises how a camera, NVR, or VMS estate enters the platform depending on what it actually exposes.

3.1 The adapter pattern

Every VMS integration in Model 3 implements one interface: `connect`, `get_cameras`, and `start_event_stream`. Table 2 shows why this matters in practice, not just in principle: three vendor payload shapes for the same underlying concept, a license-plate detection event, disagree on nearly every field name and even the timestamp encoding.

Source type	Onboarding path	Mechanism	Code path
Directly-reachable RTSP camera / NVR	Direct ingestion	Supervisor spins up one stream worker per camera, forced over TCP	Model 2
ONVIF-compliant camera or NVR	Federation, config-driven	SOAP over ONVIF Media/Device WSDL; one onboarded system can represent a multi-channel NVR	Model 3
Closed vendor VMS (Milestone / HikCentral / Dahua-class)	Federation, adapter self-registration	Vendor-specific adapter translating the VMS's own event shape into a canonical event schema	Model 3
Cloud or public REST camera API (e.g. Windy-class webcam directories)	Federation, config-driven	Generic REST + API-key adapter with dotted-path field mapping, no new code per vendor	Model 3
Bulk metadata (CSV) or manual entry, no live feed	Registry only	Camera exists as a registry row; connects to streaming or federation later if a feed becomes available	Model 1

Table 1: Onboarding path by source type.

Field	Milestone-class	HikCentral-class	Dahua-class
Plate	licensePlate	plateText	plate
Confidence	score	detectionConfidence	conf
Timestamp	ISO-8601	ISO-8601	Unix integer
Camera ID	deviceId	cameraIndex	channel
Push model	XML/SDK event push	REST subscription	WebSocket push

Table 2: Representative field-naming divergence across three vendor VMS payload shapes – the concrete shape of the interoperability problem this platform solves.

Each adapter normalises its vendor’s shape into one canonical event schema before anything downstream ever sees it. Onboarding a new *vendor that already speaks a supported protocol* – any ONVIF-compliant device, or any REST service returning a JSON array of camera-like objects – needs zero new code: the adapter registry exposes a configuration form (host, credentials, field paths) and a live *Test Connection* check before anything is persisted. Onboarding a new *protocol* still needs one new adapter class written once, which is an honest property of protocols being genuinely different things, not a gap in the framework.

3.2 One camera table, three contributors

A single **cameras** table is the shared source of truth across all three models, carrying three distinct groups of columns that different parts of the system populate:

- **Registry fields** – department, district, location, ownership, storage type, retention period. Owned by Model 1, editable by a department administrator.
- **Grid-sync fields** – codec, resolution, bitrate, stream URLs. Populated automatically by Model 2’s catalogue poller against the government camera grid’s own inventory API; Model 1 reads these but never writes them.
- **VMS linkage** – a foreign key into **vms_systems**, the join point Model 3 uses to tie a registry row to the specific federated VMS connection that reported it. **NULL** means the camera came

from the main grid or was onboarded manually; a non-null value traces it back to exactly which departmental VMS reported it.

A camera onboarded through any of these three paths shows up identically on the GIS map, in the live grid, and in gap-analysis reporting – there is no second-class camera depending on which door it came in through.

4 Registry and GIS Foundation

The registry answers the question no department currently has a consolidated answer to: what cameras actually exist, where, whose are they, and can that record be trusted. It is deliberately registry-only – it does not stream or store video – so that it can exist as a stable foundation underneath whichever streaming or federation approach a given camera ultimately uses.

4.1 Data model

- **departments** – department name and category, seed data.
- **districts** – Gujarat’s 33 administrative districts with real polygon boundaries, sourced from a public administrative-boundary dataset and matched by official Local Government Directory codes rather than by name, so the mapping is unambiguous.
- **cameras** – the core registry payload (Section 3).
- **status_history** – one row per tracked field change, written by a database trigger rather than application code, so audit logging cannot be silently skipped by a code path that forgets to call it.

District boundaries are simplified with a topology-aware algorithm run once across the whole set of 33 districts rather than polygon by polygon, so that neighbouring districts keep the exact same shared edge instead of two independently-rounded approximations of it. That matters directly for the gap-analysis computation below, which depends on those boundaries being geometrically consistent, not just visually acceptable on a map.

4.2 Coverage gap analysis

This is the one place the registry performs real spatial computation rather than serving pins on a map:

1. Every active, located camera gets a circular buffer computed as a true geodesic distance (accounting for the earth’s curvature), not a flat-plane approximation.
2. All of one district’s buffers are merged into a single coverage shape.
3. That coverage shape is subtracted from the district’s own boundary – what remains is the actual uncovered area, as a polygon, not just a percentage.
4. The uncovered area is measured against the district’s total area to produce a coverage percentage, sorted worst-covered first.

The whole computation runs as a single spatial SQL query across all 33 districts at once, letting the database’s own spatial engine parallelise the work, rather than looping over cameras in application code. This is the difference that matters once camera counts climb into the thousands: per-row overhead in application code does not scale the way a set-based database query does.

4.3 Role-based UI and audit trail

Camera markers on the map use custom, status- and department-specific iconography rather than tinted default pins, deliberately, because the map is the page an operator watches continuously during a shift; distinguishing "is anything offline right now" at a glance matters more here than almost anywhere else in the product. Bulk CSV onboarding, manual entry, and role-scoped editing all write through the same audit trigger described above, and a department administrator can only create or edit cameras within their own department – enforced at the API layer, not just hidden in the UI (Section 9).

5 Unified Viewing and Direct Video Ingestion

Model 2 answers the operational question: how does a command-centre operator watch and run analytics across dozens of independent departmental camera systems at once, without replacing any department's own VMS and without re-recording every camera's raw video into central storage.

5.1 Streaming architecture

The platform connects directly to each reachable camera over RTSP, forced over TCP for reliability, and re-serves it to operators over three paths depending on what the browser can consume: low-latency WebRTC (WHEP) for live preview, HLS for the multi-camera grid wall, and a server-side MJPEG relay as a codec-agnostic fallback. No transcoding proxy or broker sits between the camera and the first hop of this pipeline – what the camera outputs is what the ingestion worker reads.

Decoupled ingestion. Stream capture and AI inference cannot share the same event loop: decoding video over a variable-latency network would block every other request the application serves. An ingestion supervisor runs one capture worker per active camera, feeding a bounded in-memory frame queue that the inference stage reads independently. If inference falls behind, frames are dropped rather than buffered – the pipeline processes what is happening now, not a growing backlog of what happened a few seconds ago.

5.2 Selective metadata, not bulk video

Centralising raw video from a statewide camera estate does not scale – storage grows linearly with camera count and retention period regardless of whether anything interesting ever happened on a given feed. EyesOnGuj takes the opposite approach: raw frames are evaluated in memory and discarded immediately after inference. Only structured, high-value output is persisted – bounding boxes, normalised plate strings, cropped thumbnails, track identifiers, and watchlist match records. The practical consequence is that central storage grows with *incidents*, not with *camera count* or *elapsed time*, which is the only version of this that is actually compatible with an eighty-thousand-camera target.

5.3 Dual ingestion: live and forensic

The same three-stage analytics pipeline (Section 7) processes two independent input sources:

- **Live feeds** – camera workers run on demand, starting when an operator opens the live detection view and stopping when the last client disconnects, rather than running continuously in the background regardless of whether anyone is watching.
- **Forensic upload** – investigators can upload recorded footage (up to 2 GB) for the same pipeline to process at adjustable playback speed, with pause, resume and stop controls, on an isolated worker thread that never competes with live ingestion for resources.

6 VMS Federation and Middleware

Where Section 5 connects directly to a reachable stream, Model 3 is the layer that reaches departments whose cameras sit behind a closed VMS or a remote API. Architecturally the two are peers, not a pipeline: Model 3 does not sit in front of Model 2’s traffic, and Model 2’s live pipeline has no runtime dependency on Model 3 being available.

6.1 Event bus and correlation engine

Every adapter, regardless of vendor, publishes a normalised event onto a single publish/subscribe channel. One background correlation engine subscribes to that channel and runs the same pipeline for every event (Figure 2):

1. **Deduplicate** – the same plate from the same camera within a short window is collapsed to one event.
2. **Resolve the camera** – look up the registry row for this VMS and this camera’s external id; an event for a camera that was never registered is logged and dropped rather than inserted with a fabricated reference.
3. **Resolve or create the vehicle track** – derived deterministically from the normalised plate, using the *same derivation* Model 2’s own analytics pipeline uses, so a plate resolves to the same track whether Model 2’s direct pipeline or a federated adapter saw it first.
4. **Watchlist check**, before the sighting is written, so a match is known at insert time.
5. **Persist the sighting** with full raw-payload audit trail.
6. **Alert on a match**, broadcast immediately over WebSocket.
7. **Cross-system correlation** – has this same vehicle track been seen through a *different* VMS recently? If so, broadcast the full camera sequence it travelled through. This is never stored separately; it is always recomputed from the raw sightings, so it can never drift from what was actually observed.
8. **Broadcast the event** to every connected dashboard.

The event bus itself degrades gracefully: if the message broker is unreachable, publishing falls back to a bounded in-process queue with a logged warning, so a bare local environment with no broker running still works end to end rather than crashing on startup.

6.2 No separate federation schema

An earlier design gave Model 3 its own private tables. That was dropped: a camera reported by a federated VMS and a camera in the main registry were becoming two different rows for the same real-world object. Model 3 now writes into exactly the same **cameras**, **detections**, **vehicle_tracks**, and **alerts** tables that Model 1 and Model 2 already own, tagged with which VMS reported them. The practical payoff: a full multi-system route for one plate, spanning the main grid, Model 2’s own pipeline, and every federated VMS, is one query joining tables that already exist – not a second correlation mechanism that has to be kept in sync with the first.

7 AI Video Analytics Pipeline

7.1 Vehicle detection, tracking and ANPR

1. **Vehicle detection.** Generic detectors trained on Western traffic distributions underperform on Indian roads, so detection uses a model fine-tuned on Indian traffic categories specifically: cars, auto-rickshaws, motorcycles and scooters, buses, trucks, mini-trucks, and bicycles.

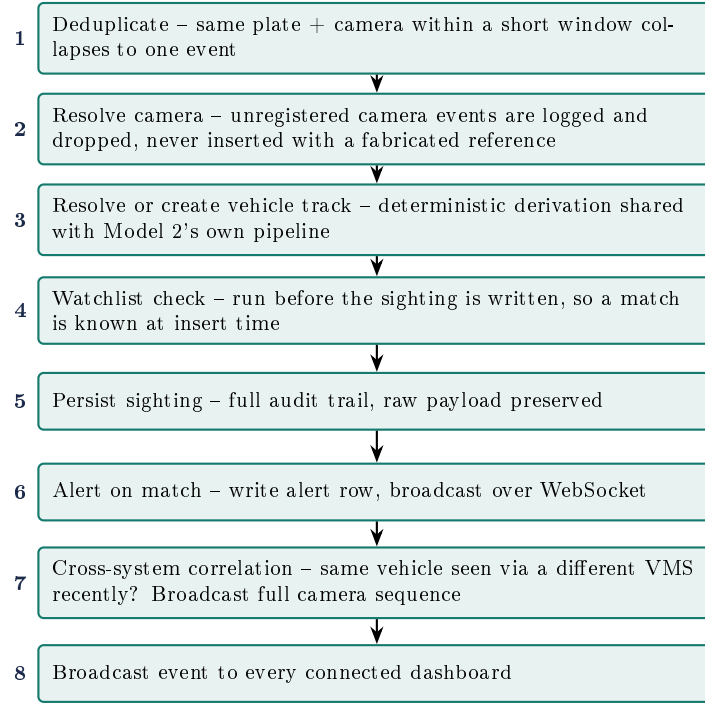


Figure 2: Correlation engine pipeline, run once per event by a single background task subscribed to the event bus.

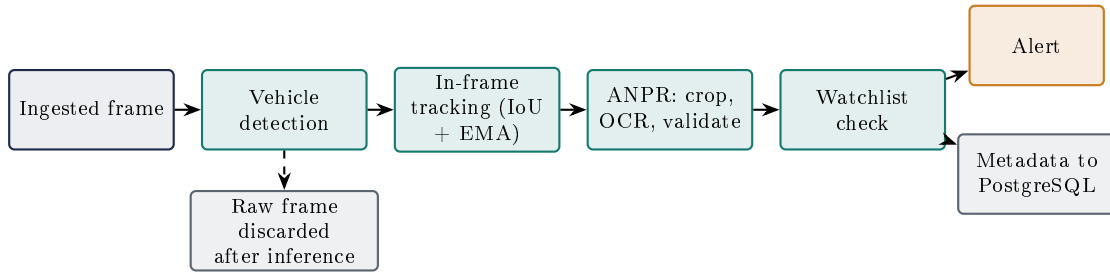


Figure 3: Three-stage vehicle analytics pipeline, run on every ingested frame.

2. **In-frame tracking.** Raw per-frame detections flicker and duplicate; a lightweight tracker assigns persistent identifiers within a single camera's field of view using spatial proximity and box overlap, with smoothing applied across frames and a minimum number of confirmed frames before a detection is promoted to a tracked object.
3. **ANPR.** Vehicle crops are routed to plate detection and OCR, and the recognised string is normalised and validated against standard Indian state-series and Bharat-series plate formats before being accepted.

Cross-camera route reconstruction – distinct from the in-frame tracking above – currently joins sightings primarily by normalised plate string and timestamp, with a fallback path for appearance-based re-identification planned for cases where OCR fails at one camera along the route. This sequencing choice is deliberate: given a working plate pipeline, a plate-string join already delivers genuine multi-camera route reconstruction, and full visual re-identification becomes an enhancement rather than a blocker for the core capability. Section 12 states this distinction plainly.

7.2 Biometric face-matching

A second, independent pipeline handles person-of-interest matching, decoupled from vehicle analytics so that a facility-security or missing-persons use case does not compete for the same code

path as traffic monitoring:

- **Registration.** A reference photo passes a five-gate quality check (face count, sharpness, pose angle within tolerance, and two further checks) before an embedding is generated, so a poor-quality reference photo is rejected at registration time rather than producing an unreliable watchlist entry later.
- **Embedding.** A 512-dimensional face embedding is stored alongside the registry record, indexed for fast approximate nearest- neighbour search.
- **Matching.** Detected faces in a video stream are embedded and compared against the active watchlist by cosine similarity; a match above the configured similarity threshold generates an alert. Non-matching embeddings and crops are discarded immediately – the platform does not retain a biometric record of anyone who is not a match against an authorised watchlist.

7.3 Evaluation discipline

Every analytics stage is evaluated on precision, recall and F1, not a single accuracy figure. This matters specifically because watchlist hits are rare events against overall traffic volume: a model that predicts "no match" on everything scores misleadingly well on raw accuracy while being operationally useless. Detection, plate-read accuracy (character-level and full-plate exact-match, reported separately, since a plate that is 90% correct character-by-character is a 0% useful match), and watchlist alerting are each measured against a held-out, hand-labelled test set rather than reported as a single headline number.

8 Watchlist Correlation and Real-Time Alerting

8.1 Data model

Table	Role
<code>vehicles_watchlist</code>	Target plates: category (stolen / wanted / blacklisted), reporting department, status
<code>persons_watchlist</code>	Target individuals with reference embeddings and quality metrics
<code>detections</code>	Every instantaneous sighting, vehicle or otherwise, with camera, timestamp, and confidence
<code>vehicle_tracks</code>	Synthesised cross-camera journeys, joining detections by resolved identity
<code>alerts / person_alerts</code>	Auditable incidents: watchlist id, severity, acknowledging operator and timestamp

Table 3: Core watchlist and alerting tables, shared across all three models.

An alert is not a boolean flag on a detection row; it is its own auditable entity with a required acknowledging operator, so an alert cannot be silently dismissed without a record of who addressed it.

8.2 Severity and notification workflow

Watchlist category	Severity	Operator-facing behaviour
Stolen	Critical	High-visibility alert, prioritised in the incident queue
Wanted	High	Flagged indicator on the live feed, prioritised queue entry
Blacklisted	Medium	Logged and surfaced, lower visual priority

Table 4: Severity derivation by watchlist category.

On a match, the alert is written and broadcast over WebSocket to every connected dashboard in well

under a second, and stays in an *unacknowledged* state until an operator acts on it. Acknowledgement is idempotent by design, so multiple control-room desks viewing the same incident feed cannot race each other into an inconsistent state.

8.3 Why plate normalisation matters here specifically

Plate strings are normalised (whitespace, punctuation and case stripped) before every watchlist comparison, because the comparison is only as reliable as the string on both sides of it. This is a small implementation detail with an outsized effect on the number that actually matters for this platform: alerting precision and recall (Section 7).

9 Security, Privacy and Auditability

9.1 Authentication and session handling

Sessions are JSON Web Tokens carried in an `httpOnly` cookie rather than browser storage, which closes off an entire class of script-based token theft; a bearer-token header is also accepted for direct API access. Passwords are hashed with bcrypt and never stored or logged in reversible form. Tokens are short-lived (an eight-hour session, matched to an operator’s shift length rather than a long-lived API credential), and the signing key has a fail-fast check: the application refuses to start in a production configuration if it detects the checked-in development default key is still active, rather than silently signing real sessions with a key that is public in the repository.

9.2 Role-based access control

Three roles, enforced at the API dependency layer rather than hidden in the UI: a department administrator with full camera and watchlist CRUD scoped to their own department, an operator with read access plus day-to-day monitoring and alert handling, and a read-only viewer explicitly excluded from audit-log access. Department scoping is layered on top of role checking, not a replacement for it – an administrator cannot create, edit, or view detail for a camera outside their own department, and bulk CSV import enforces the same rule per row rather than per file.

9.3 Login rate limiting

Failed login attempts are tracked per client-IP-and-username pair rather than per username alone, so an attacker cannot remotely lock a real user out of their own account by deliberately failing that user’s login from an arbitrary address. Five failures in five minutes locks that pair out for fifteen minutes; a successful login clears the count. This is intentionally simple, in-process state, appropriate for a single-instance deployment – Section 10 notes explicitly where this needs to move to a shared backend once the application scales horizontally.

9.4 Audit trail

Every tracked change to a camera record is written by a database trigger, not by application code calling a logging function – which means there is no code path that can mutate a camera and simply forget to log it. The same acknowledgement-based audit pattern extends to alert handling: every alert records who acknowledged it and when.

9.5 Sensitive media access

Detection crop images and face-match crops are frames pulled from live surveillance and are treated accordingly: they are not served through a generic static-file mount, which has no way to require a login before handing back a file. A dedicated route checks the requester is authenticated and that the requested path cannot escape its own storage directory before ever touching the filesystem – the one directory that actually holds sensitive imagery gets the one route that actually checks who is asking.

9.6 Transport security

TLS termination sits in front of the application at the reverse-proxy layer; the application itself speaks plain HTTP only on the private network behind it. The session cookie's `secure` flag tracks the same production/debug switch the rest of the configuration uses, so it cannot drift out of sync with whether TLS is actually in front of it.

9.7 What is documented here but not yet built

Stated plainly, in the same spirit the rest of this project's engineering documentation already uses for unfinished work, rather than left implicit:

- **At-rest encryption** for watchlist and detection tables specifically – flagged as a pre-production requirement, not implemented in the current build.
- **Encryption for stored VMS credentials.** Config-driven VMS onboarding can hold real credentials (an API key, a device username and password); today that configuration is stored as plain structured data, readable by anything with database access. The intended fix is application-layer encryption with the key held outside the database, or moving the secret itself into a proper secrets manager and storing only a reference here – before this holds a real department's credentials rather than a demo key.
- **Network segmentation** between the VMS-ingestion network and the public dashboard network – described in the deployment architecture (Section 10), not yet physically separated in the current single-host deployment.
- **Edge-level API gateway / rate limiting**, distinct from the login-specific limiter above – the statewide-scale answer to abusive traffic generally, not needed at pilot scale.

10 Deployment Architecture and Statewide Scale Plan

10.1 Current deployment

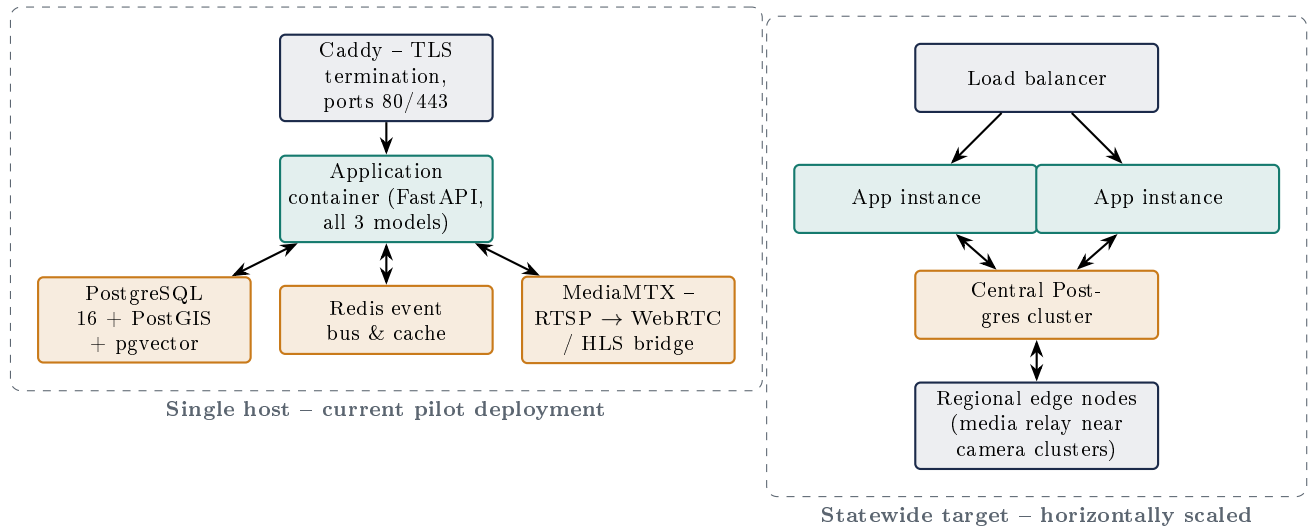


Figure 4: Current single-host deployment topology.

The current deployment is one reverse proxy handling TLS termination in front of one application container, backed by PostgreSQL with the PostGIS and pgvector extensions, a Redis instance serving as the federation event bus and cache, and a dedicated media server handling RTSP-to-WebRTC/HLS bridging. This is intentionally the simplest topology that is still production-shaped:

everything that would need to change to scale out is isolated behind an interface (the database connection, the event bus, the media relay) rather than hardwired into application logic.

10.2 Path to statewide scale

Table 5 lays out what changes, and what deliberately does not, moving from a pilot deployment toward the eighty-thousand-camera target the state has described.

Dimension	Pilot (today)	Statewide target
Compute	Single application container	Central application tier behind a load balancer, horizontally scaled; regional edge nodes running the media relay close to camera clusters to cut backhaul over 1000km site spans
AI inference	Single-GPU pipeline, throughput measured directly rather than estimated	GPU capacity extrapolated from the pilot’s own measured frames-per-second, not a guessed figure
Storage	Single PostgreSQL instance	Tiered hot (recent, SSD/NVMe) / warm (departmental retention window, HDD or object storage) / cold (beyond retention, self-hosted S3-compatible), honouring each department’s own stated retention period rather than one statewide policy
Bandwidth	Full-resolution RTSP to a nearby relay	Edge-side inference so only detection metadata and a cropped thumbnail cross constrained long-haul links, not full video
Login rate limiting	In-process, single instance	Moves to a shared backend once multiple application instances exist behind a load balancer, so separate processes don’t each keep an independent, ineffective counter
Disaster recovery	Single host, no live DR	Regional replicas and scheduled backups with a stated RTO/RPO target

Table 5: Scaling dimensions from pilot to statewide deployment.

10.3 Cost posture

The stack is fully open source end to end; there is no proprietary VMS SDK dependency in the core platform. The concrete cost argument against a fully centralised commercial-VMS approach is licensing: this architecture’s licensing cost is zero, and its main real cost drivers at scale are GPU inference capacity and cold-storage volume, both of which are quantifiable once real pipeline throughput numbers exist, rather than estimated in the abstract.

10.4 Phased rollout

Phase	Camera count	Focus
Pilot	Tens of cameras	Registry, direct ingestion, core analytics and alerting proven end to end
Regional	Low thousands	Federation adapters exercised against real departmental VMS estates; edge relay nodes in
Statewide	~80,000	Full storage tiering, horizontal application scaling, regional edge compute

11 Assumptions and Information Required from Departments

Each participating department needs to supply the following before its cameras can be onboarded, summarised in Table 6.

Information	Why it's needed
VMS vendor and protocol (or confirmation of direct RTSP reachability)	Determines onboarding path – direct ingestion vs. a specific federation adapter
Network reachability and firewall rules	Whether the platform can reach the camera or VMS directly, or requires a department-side relay
Credentials (RTSP, ONVIF, or VMS API)	Required for the federation adapter's config-driven onboarding and its live test-connection check
Camera inventory (location, ownership, retention period)	Seeds the registry record; retention period specifically feeds the storage-tiering plan in Section 10
Existing watchlist data (if any)	Feeds the department-specific watchlist categories used in alert correlation

Table 6: Information required from a department before onboarding.

Explicit scope decision – analog cameras. The current build targets IP and ONVIF-reachable cameras only. Digitising analog feeds needs encoder hardware that has not been part of this build's scope; the intended path is an encoder/gateway layer feeding into the same ingestion adapters described in Section 3, not a separate architecture. This is a bounded, well-understood addition and is named here rather than left for someone to discover the hard way.

12 Implementation Status

This section states, plainly and without hedging, what is fully built, what is partially built, and what is designed but not yet implemented. The rest of this document describes the architecture; this section describes how much of it is real code today.

12.1 Testing approach

The registry and federation test suites run against a real PostgreSQL and PostGIS instance rather than a mocked or SQLite substitute, because the functionality under test – geodesic spatial computation, database triggers, composite uniqueness constraints – has no meaningful SQLite equivalent. Each test runs inside its own transaction and is rolled back afterward, so the suite can freely exercise create/update/delete paths through the real API without leaking state between runs or needing a reseed between them. The ONVIF adapter specifically has been verified against a live ONVIF-capable device, not only against a mock, which caught a real asynchronous-factory bug that a more permissive mock had missed; the REST/API-key adapter has similarly been verified end to end against a real third-party API, not only against a synthetic test harness.

13 Appendix

13.1 Glossary

13.2 Data model overview

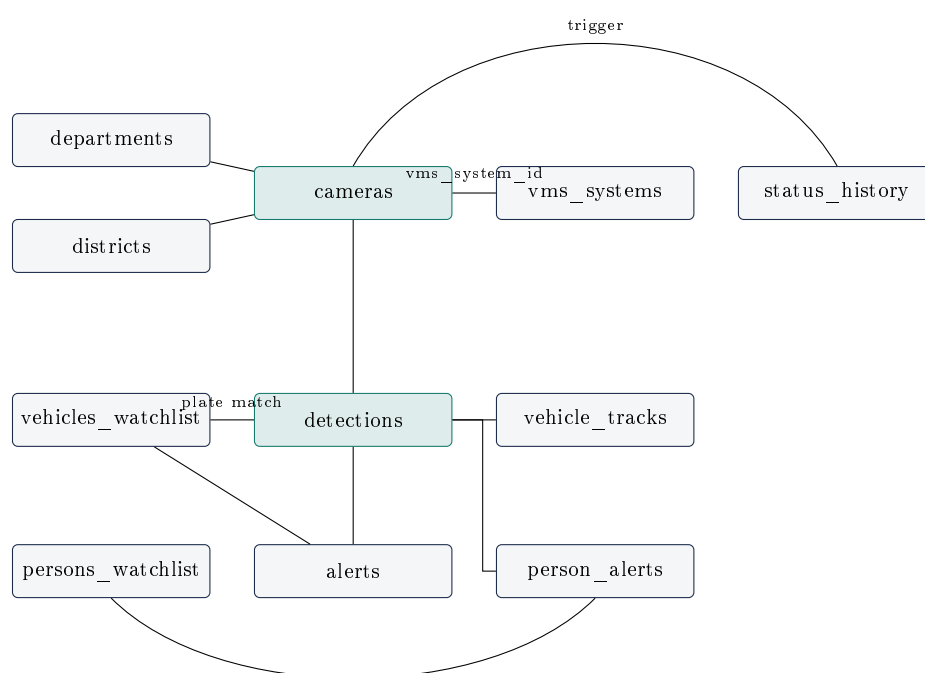


Figure 5: Simplified entity relationships across the shared schema. Full DDL is maintained separately in the codebase.

Capability	Detail	Status
Camera registry & CRUD	Full create/read/update/soft-delete, department-scoped RBAC	IMPLEMENTED
Bulk CSV onboarding	Row-level validation and partial-success reporting	IMPLEMENTED
GIS map & coverage gap analysis	Real PostGIS geodesic computation, not display-only	IMPLEMENTED
Audit trail	Trigger-written, camera changes and alert acknowledgement	IMPLEMENTED
Direct RTSP ingestion & live grid	WebRTC/HLS/MJPEG relay, decoupled worker pipeline	IMPLEMENTED
Vehicle detection	Indian-traffic fine-tuned model, evaluated on precision/recall/F1	IMPLEMENTED
ANPR (plate read)	Plate crop, OCR, Indian state and Bharat-series format validation	IMPLEMENTED
Watchlist correlation & alerting	Live detection-to-watchlist match, severity assignment, WebSocket alert broadcast, operator acknowledgement	IMPLEMENTED
Cross-camera route reconstruction	Plate-string-and-timestamp join across cameras and VMS sources	IMPLEMENTED
Appearance-based re-identification fallback	For cases where OCR fails at one camera along a route	PLANNED
Face registration & embedding	Five-gate quality pipeline, 512-d embedding, vector index	IMPLEMENTED
Live face-matching against watchlist	Face extraction from live/recorded video, cosine-similarity match, alert generation	IMPLEMENTED
VMS federation adapters (ONVIF, REST/API-key, three simulated vendor adapters)	Adapter registry, config-driven onboarding, live test-connection check	IMPLEMENTED
Cross-system correlation engine	Dedup, resolve, watchlist check, cross-VMS route stitching	IMPLEMENTED
At-rest encryption (watchlist/detection tables)	See Section 9	PLANNED
VMS credential encryption	See Section 9	PLANNED
Network segmentation	See Section 9	PLANNED
Analog camera ingestion	See Section 11	PLANNED
Live disaster recovery	Regional replication, tested failover	PLANNED

Table 7: Implementation status by capability.

Term	Meaning
ANPR	Automatic Number Plate Recognition
FRS	Facial Recognition System
GIS	Geographic Information System
HLS	HTTP Live Streaming
NVR	Network Video Recorder
ONVIF	Open Network Video Interface Forum (an open camera/NVR protocol standard)
RBAC	Role-Based Access Control
RTSP	Real Time Streaming Protocol
VMS	Video Management System
WHEP	WebRTC-HTTP Egress Protocol